Journal of Network and Computer Applications 134 (2019) 26–39

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

Cloud storage auditing with deduplication supporting different security levels according to data popularity

Huiying Hou [a,b], Jia Yu [a,c,d,*], Rong Hao [a]

[a] College of Computer Science and Technology, Qingdao University, 266071, Qingdao, China
[b] College of Computer Science and Technology, Fudan University, 200433, Shanghai, China
[c] Jiangsu Key Laboratory of Big Data Security and Intelligent Processing, Nanjing University of Posts and Telecommunications, 210023, Nanjing, China
[d] State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, 100093, Beijing, China

ARTICLE INFO

Keywords:
Cloud storage auditing
Data deduplication
Semantic security
Data popularity

ABSTRACT

The public cloud storage auditing with deduplication is proposed to check the integrity of cloud data under the condition that the cloud stores only a single copy of the same file from different users. To the best of our knowledge, the existing schemes about cloud storage auditing with deduplication cannot support semantic security for cloud data. The recent data breach incidents have led to an increased demand for the security of encryption schemes. Under the circumstances, we consider to provide different security levels according to the popularity of cloud data. We use the semantic secure encryption to encrypt the unpopular data (private data) to realize semantic security and use convergent encryption to encrypt popular data to realize ciphertext deduplication. However, there exists a big challenge for cloud storage auditing when data popularity changes. Because encryption algorithms are different for popular data and unpopular data, the corresponding ciphertext will have to change once data popularity changes. The old authenticators cannot be valid for the integrity checking any longer after ciphertext changes. In order to overcome this challenge, we explore the numerical relationship between old authenticators and new ones. In our designed scheme, it is not necessary for users to be online for doing extra computation when data popularity changes. The cloud can perform the task of authentications transforming to ensure that the cloud storage auditing still smoothly runs. By detailed security Proof and performance analysis, we show that the proposed scheme is secure and efficient.

1. Introduction

On account of providing scalable, low-cost, and location-independent storage services, the cloud storage grows in popularity. The number of users serviced by the cloud has increased dramatically (Aujla et al., 2018; Wazid et al., 2018). To gain more profit, the most crucial thing is to improve the storage efficiency for cloud. Since the different users often send the same data to the cloud, it will incur the extended storage costs. The 75% of recent digital data is duplicated according to a survey from EMC (Gantz and Reinsel, 2010). So, the cloud goes in for how to store only a single copy in the above circumstances. In order to realize ciphertext deduplication, the data needs to be encrypted by convergent encryption to generate the same ciphertext for the same data. However, convergent encryption cannot provide semantic security. The recent data breach incidents have led to an increased demand for the security of encryption schemes. If users adopt semantic secure encryption schemes to encrypt user data, the same data will be encrypted into different ciphertexts. It means the cloud cannot eliminate the duplicated data based on ciphertext.

To solve this problem, the outsourced data is divided into two sorts: popular and unpopular. If the owner's number of the file goes beyond the threshold value, the file is popular. Otherwise, the file is unpopular. An entity called as indexing service (IS) (Stanek et al., 2014) is used to record how many distinct users have the same file to prevent malicious cloud cheating on file popularity. Based on the records from IS, the cloud provides semantically security for unpopular data but weaker security for popular data. In this case, the cloud can provide better storage and bandwidth under protecting the private data. In other words, IS is used to judge the popularity of files and classify them in ciphertext deduplication system that supports popularity. Therefore, entity IS is

* Corresponding author. College of Computer Science and Technology, Qingdao University, 266071, Qingdao, China.
E-mail address: qduyujia@gmail.com (J. Yu).

https://doi.org/10.1016/j.jnca.2019.02.015
Received 18 November 2018; Received in revised form 6 February 2019; Accepted 19 February 2019
Available online 23 February 2019

Table 1
Functional comparison among the existing schemes and our scheme.

Function	Shacham and Waters (2008)	Wang et al. (2016)	Douceur (2002)	Bellare and Goldreich (1992)	Wazid et al. (2018)	Zheng and Xu (2012)	Pietro and Somiotti (2012)	Halevi et al. (2011)	our proposed scheme
Cloud storage auditing	✓	✓	✓	×	×	✓	✓	✓	✓
Ciphertext Deduplication	×	×	×	✓	✓	✓	✓	✓	✓
Security level	ss	ss	ss	Nss	Nss	Nss	Nss	Nss	ss

[1]ss: semantic security.
[2]Nss: Non-semantic security.

required in this system. If there is no Idp in the system, it may suffer sybil attack (Douceur, 2002). The Idp is also necessary in the system.

In order to verify the integrity of the data stored in public cloud, many cloud storage auditing schemes are proposed. (Ateniese et al., 2007; Halevi et al., 2011; Shacham and Waters, 2008). Recently, how to efficiently check the integrity of the duplicated cloud data has become a hot research topic. Some people research identity-based cloud storage auditing (Zhang et al., 2018a, 2018b; Shen et al., 2019; Wang et al., 2016). In order to reduce user's computational burden, some researchers study the public cloud storage auditing (Wang et al., 2011; Ateniese et al., 2008; Shen et al., 2017a; Liu et al., 2013; Yu et al., 2016; Yuan and Yu, 2014) in which the user can delegate auditing task to a trust third party auditor(TPA). In order to protect user's privacy, some explore the privacy-preserving auditing scheme (Wang et al., 2010, 2013; Shen et al., 2017b; Yang et al., 2016). In order to resist key-exposure, some research the cloud storage auditing with key-exposure resistance (Yu et al., 2015; Yu and Wang, 2017). Multiple cloud storage auditing schemes with deduplication have been proposed (Hou et al., 2018; Pietro and Sorniotti, 2012; Zheng and Xu, 2012; Alkhojandi and Miri, 2015; Yuan and Yu, 2013; Li et al., 2016). However, none of them supports semantic security for private data. They all adopt convergent encryption algorithms or modified convergent encryption algorithms to encrypt the user data. These encryption algorithms cannot provide semantic security. It may have no influence for popular data. But the security of these encryption algorithms is not enough for unpopular data (Stanek et al., 2014) distinctly. The popular data is the outsourced data uploaded by many users, such as a popular movie or song. The unpopular data is the outsourced data uploaded by few users, such as a secret scientific paper. If we directly adopt the semantic secure encryption to encrypt all files, the data deduplication is impossible. Clearly, the cloud cannot achieve data deduplication without knowing the decryption key. In order to achieve data deduplication and semantic security for unpopular data simultaneously, in this paper, we use the semantic secure encryption to encrypt the unpopular data (private data) to realize semantic security and use convergent encryption to encrypt unpopular data to realize ciphertext deduplication (see Table 1).

There still exists some challenges to design a cloud storage auditing scheme with deduplication supporting different security levels according to data popularity. The main challenge is from data popularity changing. Concretely speaking, the unpopular file may become a popular file with increasing of its owner's number. Because encryption algorithms are different for popular data and unpopular data, the corresponding ciphertext will have to change once data popularity changes. It means the old authenticators based on the old ciphertext do not support the integrity checking for the new ciphertext any more. In order to realize the integrity auditing in this scenario, we firstly try to apply a straightforward method to solve this problem. The data owner downloads all his data previously stored in the cloud, produces new authenticators for the corresponding new ciphertext, and then uploads the new ciphertext and authenticators. However, it will bring big computation and communication burden on the data owner. Besides, all the data owners are required to be always online. Secondly, we try to introduce a third party to produce new authenticators for the data owners. It can release user's computation and communication burden. However, it will increase the complexity of the system and even cause some new security issues. Therefore, it is also not a good solution to the problem. Finally, we give our core scheme to well solve this problem.

Contributions. To achieve ciphertext deduplication, the data needs to be encrypted by convergent encryption to generate the same ciphertext for the same data. However, convergent encryption cannot provide semantic security. In other words, the security level of convergent encryption is relatively low. The symmetric encryption can provide semantic security. But it will make the same data be encrypted into different ciphertexts. As a result, the cloud cannot realize the deduplication based on the ciphertext. To achieve a balance between security and storage efficiency, we adopt the convergent threshold cryptosys-

tem. The convergent threshold cryptosystem is an asymmetric cryptographic scheme. Using the public key to encrypt and the private key to decrypt. The private key is divided into n secret shares and distributed to n users. In this paper, the data is encrypted by convergent encryption firstly. On this basis, the convergent threshold cryptosystem is used for encryption. Assume the threshold is t. When the number of users with the same data reaches t(the unpopular data become popular), the cloud can recover the decrypted private key and obtain the ciphertext of convergent encryption. Then the cloud can achieve ciphertext deduplication. In order to audit the integrity of cloud data without downloading the entire data file, integrity auditing schemes are proposed. The user generates the ciphertext and the corresponding authenticators before uploading them to the cloud. The authenticators are generated by the secret key of the user, so the cloud cannot forge a valid authenticator. When the data stored in the cloud matches the authenticators, we think the cloud stores the complete user data.

In this paper, we explore how to realize cloud storage auditing with deduplication supporting different security levels according to data popularity and propose the first scheme satisfying this property. The proposed scheme achieves the semantic security for unpopular data and the ciphertext deduplication for popular data simultaneously. In addition, the third party auditor (TPA) still can audit the integrity of cloud data even if data popularity changes. In order to realize our goal, we explore the numerical relationship between old and new authenticators. The cloud can produce new authenticators on behalf of data owners under the condition that he does not know any private information of data owners. In this way, data owners do not need to produce new authenticators by themselves and be always online. We give the formal Proof of our scheme's the security, and demonstrate it's performance via concrete experiments. The asymptotic analysis results show that the proposed scheme has nice security and efficiency.

Organization. The rest paper is organized as follows: The related work shows in section 2. In section 3, we present the system model, definition, security model and preliminaries. We follow with a description of our scheme in section 4. Then, we give efficiency evaluation and security theorem in section 5. We give the conclusion in section 6.

2. Related work

In this section, we review the related work about integrity auditing and secure deduplication respectively.

2.1. Integrity auditing

In 2007, Ateniese et al. (2007) first defined the Provable Data Possession (PDP) and presented a PDP scheme. In this scheme, the data owner can audit the integrity of cloud data without downloading the entire data file. To realize PDP on dynamic cloud data, Ateniese et al. (2008) proposed another PDP scheme, which supported all of dynamic operations except insertion operation. Shen et al. (2017a) proposed a dynamic PDP scheme which supports full dynamic operations. To reduce the computation burden on the user side, the public auditing schemes (Liu et al., 2013; Shen et al., 2017b; Wang et al., 2010, 2016; Yang et al., 2016; Yu et al., 2015, 2016; Yu and Wang, 2017; Yuan and Yu, 2014) were proposed to allow a Third Party Auditor (TPA) on behalf of the data owner to audit the integrity of their cloud data.

2.2. Secure deduplication

Deduplication technique allows the cloud store only a single copy for each file, regardless of how many users ever uploaded that file. When a user uploads a file to cloud, the cloud first judges whether the hash value of this file has already been stored. If it is, the cloud builds a link between the file and the owner; Otherwise, not. However, deduplication will cause a security problem. The problem is that the cloud will build a link between the file and the user who has the hash value but actually does not possess this file. In order to solve this problem, Halevi et al. (2011) firstly presented a Proof of ownership scheme to achieve secure deduplication. Pietro et al. (Pietro and Sorniotti, 2012) proposed an improved secure proof of ownership scheme that reduced the computation burden of the data owner. In order to perform integrity auditing and deduplication simultaneously, some public auditing schemes with deduplication have been proposed (Alkhojandi and Miri, 2015; Yuan and Yu, 2013; Li et al., 2016). However, all previous schemes about cloud storage auditing with deduplication did not consider the different security requirements for the different kinds of outsourced data. These schemes cannot provide semantic security for any outsourced data. Thus we consider to solve this problem in this paper.

3. Formulation and preliminaries

3.1. System model

The system model consists of five kinds of different entities: The user, the identity provider (IdP), the cloud, the indexing service (IS) and the third party auditor(TPA) (see Fig. 1). User has a large amount of data to be stored on the cloud. User can be either an individual consumer or a commercial organization. The identity provider (IdP) issues an identification credential to user when he first joins the system. The IdP is used to thwart sybil attacks by allowing that user to register only once. We regard this as an orthogonal problem which many effective solutions are pointed out in (Douceur, 2002). The cloud virtualizes the resources according to the user's requirement and exposes himself as storage pools. To be specific, the user may buy or rent spaces from cloud. To improve the storage efficiency and gain more benefits, the cloud willing to eliminate the deduplicated data. The indexing service (IS) maintains a record of how many distinct users have uploaded one same file and issues a unique file identifier for an uploaded cloud file. The third party auditor (TPA) periodically audits whether the cloud data is stored correctly or not. From checking the data possession Proof from the cloud, the TPA returns the auditing results to the user.

3.2. Definition and security model

(1) The definition

Definition 1. A cloud storage auditing with deduplication supporting different security levels according to data popularity is composed by seven algorithms (*Setup, Join, Upload, AuthGen, PopulartityChange, ProofGen, ProofVerify*):

1) $Setup(1^{\kappa}) \rightarrow (pk, \{x_i\}_{i=0}^{n-1})$: It inputs a security parameter κ. Then it outputs the public key pk, n shares $\{x_i\}_{i=0}^{n-1}$ of the secret key. This algorithm is a probabilistic algorithm and executed by the identity provider (IdP).
2) $Join(\text{theidentityoftheuser}U_i) \rightarrow$ (**Ui**, x_i): This algorithm is a probabilistic algorithm which takes the identity of the user *Ui.* It outputs a secret key share x_i and a user identifier **Ui.** This algorithm is executed by the identity provider (IdP).
3) $Upload(F) \rightarrow$ (execute *Upload.Popular* or *Upload.Unpopular*): This algorithm is a probabilistic algorithm which takes the file F as input. The user communicates with the Indexing Service (IS). According to IS's response, the user selects to execute sub-algorithms *Upload.Popular* or *Upload.Unpopular.* This algorithm is executed by the user.
4) $AuthGen(C = \{c_1, c_2, \ldots, c_n\}) \rightarrow (k_{tag}, v, \{T_i\}_{1 \le i \le n}, \{{u_1}^{k_{tag}}, {u_2}^{k_{tag}}, \ldots, {u_s}^{k_{tag}}\}, SSig, ssk, P_{ssk})$: It inputs the ciphertext of the file F. It outputs the key k_{tag} that used to generate authenticators, the key $v \leftarrow g^{k_{tag}}$, the set $\{{u_1}^{k_{tag}}, {u_2}^{k_{tag}}, \ldots, {u_s}^{k_{tag}}\}$ of authenticators $\{T_i\}_{1 \le i \le n}$, the file

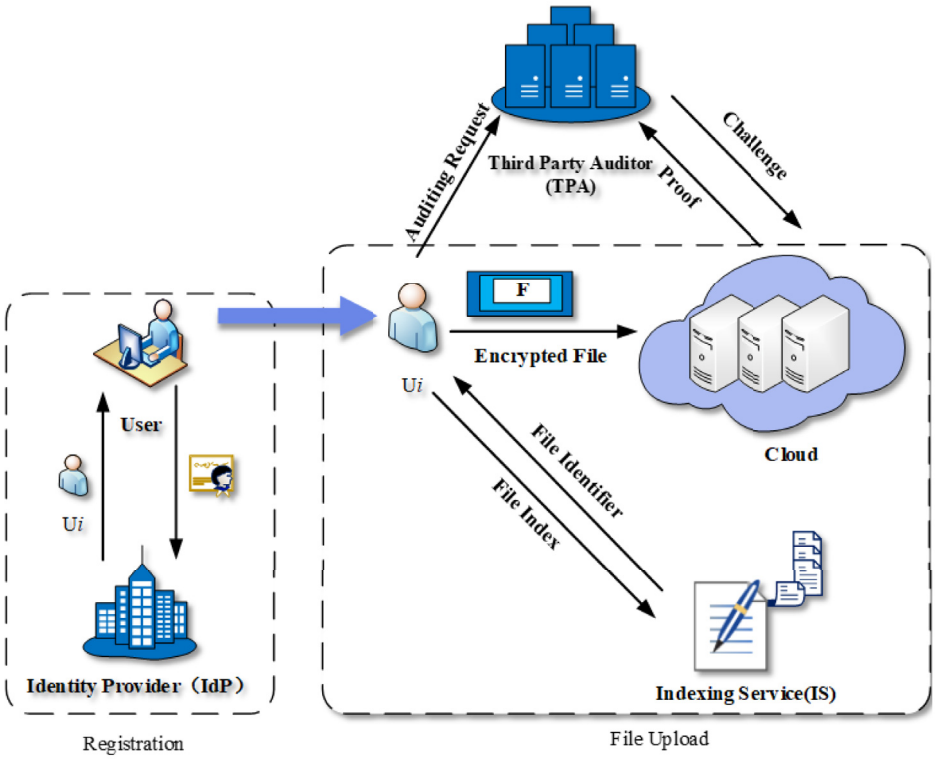


Fig. 1. The framework of the system model.

tag *SSig*, the signing key *ssk* and the verification key *Pssk*. This algorithm is a probabilistic algorithm and executed by the user.

5) *PopularityChange* $(C) \rightarrow (\{T_i'\}_{1\le i\le n})$: It inputs the ciphertext of the file *F*. It outputs the new authenticators $\{T_i'\}_{1\le i\le n}$ for all blocks of file *F*. This algorithm is a probabilistic algorithm and executed by the cloud.
6) *ProofGen*(*chal*, $\{c_i\}_{1\le i\le n}$, $\{T_i\}_{1\le i\le n}$): It inputs the auditing challenge *chal*, ciphertext of file blocks $\{c_i\}_{1\le i\le n}$ and authenticators $\{T_i\}_{1\le i\le n}$. Then it outputs an auditing Proof *P* that is used to prove the cloud actually stores the user file completely. This algorithm is a probabilistic and executed by cloud.
7) *ProofVerify* $(\text{chal}, \text{P}, \text{v}) \rightarrow$ (TrueorFalse): It inputs the auditing challenge *chal*, the auditing Proof *P*, and the verification key *v*. It outputs *True* or *False*. This algorithm is a deterministic algorithm and executed by TPA.

(2) Security Model

In order to formally the security of the integrity auditing, we define the security model. We define one games interacted between an adversary *A* and a challenger *C* to capture the security of integrity auditing. This game includes the following four phases.

Definition 2. Supposing there exists an efficient knowledge extractor, which can capture the challenged blocks with high probability, whenever an adversary in the above described game can construct a valid Proof *P* with non-negligible probability, we say an auditing scheme is secure. In addition, we give the following definition to show the cloud correctly stores the data blocks unchallenged with high probability.

Definition 3. For ρ corrupted blocks, if we can detect the corrupted blocks with at least δ probability, then say an auditing scheme is (ρ, δ) detectable $(0 < \rho, \delta < 1)$.

1) **Setup phase** The challenger *C* randomly selects a secret key $k_{tag} \leftarrow Z_p^*$. Then, *C* computes and publishes the key $v \leftarrow g^{k_{tag}}$. *C* computes $\{u_1{}^{k_{tag}}, u_2{}^{k_{tag}}, \ldots, u_s{}^{k_{tag}}\}$, the signing key *ssk* and the verification key *Pssk*. Finally, *C* sends *v*, $\{u_1{}^{k_{tag}}, u_2{}^{k_{tag}}, \ldots, u_s{}^{k_{tag}}\}$, and *Pssk* to *A*. The challenger *C* stores *ktag* and *ssk* locally.
2) **Query phase** The adversary *A* selects and sends a line of blocks $m_1, \ldots, m_d$ as his inquiries to the challenger *C* adaptively. Then *C* responses the corresponding authenticators $T_1, T_2, \ldots, T_d$ to *A*.
3) **Challenge Phase** In this phase, the challenger *C* submits the challenge *chal* to the adversary.
4) **Forgery phase** Then, the adversary *A* outputs a Proof *P*. If the proof *P* passes the verification algorithm *ProofVerify (chal, P, v)*, then the adversary is the winner. From the above security model, we can know that the adversary wins the game, if he can construct a valid proof for the challenge message. Via knowledge extractor introduced in (Bellare and Goldreich, 1992), the security of auditing scheme can define as follows.

3.3. Preliminaries

1. **Bilinear Map** Let G_1 and G_2 be two multiplicative groups of the same large prime order *p*. We say a map $\hat{e} : G_1 \times G_1 \rightarrow G_2$ is a bilinear map if it satisfies the following properties:
   1) Bilinearity: $\forall g_1, g_2 \in G_1$ and $\forall a, b \in Z_p^*$, $\hat{e}(g_1^a, g_2^b) = \hat{e}(g_1, g_2)^{ab}$.
   2) Non-degeneracy: $\hat{e}(g_1, g_2) \neq 1$, where g_1, g_2 are generators of G_1.
   3) Computability: for all $g_1, g_2 \in G_1$, there exists an efficient algorithm to compute $\hat{e}(g_1, g_2)$.
2. **Computational Diffie-Hellman (CDH) Problem** Given $g, g^a, g^b \in G_1$, where g is a generator of multiplicative group G_1 with order *p*. The Computational Diffie-Hellman problem is that computing g^{ab} for unknown $a, b \in Z_p^*$. If an algorithm *IG*

inputs a security parameter k, and outputs the description of two groups G_1, G_2 and a bilinear map $\hat{e} : G_1 \times G_1 \rightarrow G_2$ satisfying that the CDH problem in G_1 is difficult, it is called CDH parameter generator.

3. **Discrete Logarithm (DL) Problem** Given $g, g^x \in G_1 (x \in Z_p^*)$, where g is the generator of multiplicative group G_1 with order p. The Discrete Logarithm problem is that computing x.

3.4. Building blocks

- **Symmetric Cryptosystems and Convergent Cryptosystem**
  - A symmetric cryptosystem ε is defined as a tuple (*K, E, D*) of probabilistic polynomial time algorithms.
    1. $\varepsilon.K$ inputs a security parameter κ and outputs a random secret key k.
    2. $\varepsilon.E$ inputs a secret key k and a message m. It outputs a ciphertext c.
    3. $\varepsilon.D$ inputs the secret key k and the ciphertext c. It outputs the message m.
  - A convergent encryption scheme ε_C (also named message-locked encryption scheme) is defined as a tuple (*K, E, D*). The main difference between convergent encryption ε_C and symmetric encryption ε is that ε_C is not probabilistic. This is because the key generated by $\varepsilon_C.K$ is a deterministic function of the plaintext message m.
    1. $\varepsilon_C.K$ inputs a security parameter κ and a message m. It outputs a secret key k_m. The same massage will generate the same secret key.
    2. $\varepsilon_C.E$ inputs secret key k_m and a message m. It outputs a ciphertext c.
    3. $\varepsilon_C.D$ inputs the secret key k_m and the ciphertext c. It outputs the message m.
- **Threshold Cryptosystems** Threshold cryptosystems offer the ability to share the secret key among w authorized users, such that any t of them can recover this key efficiently. According to the security properties of threshold cryptosystems, it is computationally infeasible to recover the key with fewer than t users. In this paper, we use threshold public-key cryptosystem, in which the public key is used to encrypt the massage and any t authorized users can decrypt the corresponding message ciphertext. A threshold public-key cryptosystem ε_t is defined as an algorithm tuple (*Setup, Encrypt, Dshare, Decrypt*).
  1. $\varepsilon_t.Setup(\kappa, w, t) \rightarrow (pk, sk, S)$: It inputs a security parameter, the number w of authorized users, and a threshold value t. It outputs the public key of the system pk, the corresponding private key sk and a set $S = \{(r_i, sk_i)\}_{i=0}^{w-1}$ composed by w shares of sk such that any set t shares can be used to reconstruct sk through polynomial interpolation. Note that r_i is the preimage of sk_i under the aforementioned polynomial.
  2. $\varepsilon_t.Encrypt(pk, m) \rightarrow (c)$: It inputs a public key pk and a message m. It outputs the corresponding ciphertext c.
  3. $\varepsilon_t.Dshare(r_i, sk_i, m) \rightarrow (r_i, ds_i)$: It inputs a message m and a key share sk_i with its preimage r_i. It outputs a decryption share ds_i and the corresponding r_i.
  4. $\varepsilon_t, Decrypt(c, S_t) \rightarrow (m)$: It inputs a ciphertext c and a set $S_t = \{(r_i, ds_i)\}_{i=0}^{t-1}$ composed by t pairs of decryption shares and the corresponding r_i. It outputs the plaintext message m.
- **A Convergent Threshold Cryptosystem** A convergent threshold cryptosystem ε_μ is defined as an algorithm tuple (*Setup, Encrypt, Dshare, Decrypt*).

In this paper, we use these two cyclic multiplicative groups G_1 and G_2 as Symmetric Extensible Diffie Hellman Assumption (SXDH) groups. It means that there is no efficient distortion map $\phi : G_1 \rightarrow G_2$, or $\phi : G_2 \rightarrow G_1$. Let g be the generator of G_1.

$$\prod_{(r_i, sk_i) \in S_t} sk_i^{\lambda_{0,r_i}^{S_t}} = \prod_{(r_i, x_i) \in S'_t} H_1(m)^{x_i \lambda_{0,r_i}^{S_t}} = H_1(m)^{\sum_{(r_i, x_i) \in S'_t} x_i \lambda_{0,r_i}^{S_t}} = H_1(m)^x$$

where $\lambda_{0,r_i}^{S_t}$ are the Lagrange coefficients of the polynomial with interpolation points from the set $S'_t = \{(r_i, x_i)\}_{i=0}^{t-1}$. Then it sets $\widehat{E} = \widehat{e}(H_1(m)^x, C_2) = \widehat{e}(H_1(m)^x, g^r) = \widehat{e}(H_1(m), g^x)^r = \widehat{e}(H_1(m), g_{pub})^r$. Finally, it outputs the message $m = C_1 \oplus H_2(\widehat{E})$.

1. $\varepsilon_\mu.Setup(\kappa, w, t) \rightarrow (pk, sk, S)$: It takes a security parameter κ, the number w of authorized users, and a threshold value t as input. At first, it randomly selects a secret $x \leftarrow Z_p^*$. Let $x_i (i = 0, \ldots, w-1)$ be w shares of x such that any t shares can be used to reconstruct x through polynomial interpolation. Then, let $g_{pub} \leftarrow g^x$. Finally, let $H_1 : \{0,1\}^* \rightarrow G_1$ and $H_2 : G_2 \rightarrow \{0,1\}^l$ be two cryptographic hash functions. It outputs the public key $pk = \{p, G_1, G_2, \widehat{e}, H_1, H_2, g, g_{pub}\}$, the secret key $sk = x$, and the set of w pairs $\{(r_i, x_i)\}_{i=0}^{n-1}$, where r_i is the preimage of x_i under the aforementioned polynomial.
2. $\varepsilon_\mu.Encrypt(pk, m) \rightarrow (C)$: It takes a public key pk and a message m as input. At first, it selects $r \leftarrow Z_p^*$. Then it computes $E \leftarrow \widehat{e}(H_1(m), g_{pub})^r$. Finally, it computes $C_1 \leftarrow H_2(E) \oplus m$ and $C_2 \leftarrow g^r$. It outputs the ciphertext $C = (C_1, C_2)$.
3. $\varepsilon_\mu.Dshare(r_i, x_i, m) \rightarrow (r_i, sk_i)$: It takes a message m and a key share x_i with its preimage r_i as input. It computes $sk_i = H_1(m)^{x_i}$ and outputs pairs (r_i, sk_i).
4. $\varepsilon_\mu, Decrypt(C, S_t) \rightarrow (m)$: It takes a ciphertext $C = (C_1, C_2)$ and a set $S_t = \{(r_i, sk_i)\}_{i=0}^{t-1}$ of t decryption shares as input. At first, it computes

4. Our proposed scheme

In this section, we firstly show two basic solutions to the problem of how to realize cloud storage auditing with deduplication supporting different security levels according to data popularity. The first is a naive solution, which will consume a lot of computational and communicational resources of the users. The second is a slightly better solution, which can solve this problem but maybe incur some security issue. They are both impractical in realistic setting. Finally, we give our core scheme that is much more efficient than previous solutions.

4.1. Naive solution

Once the popularity of cloud data changes, the cloud informs all data owners to update their authenticators because the encrypted secret key and the corresponding ciphertext will change. Each of data owner downloads all his previously stored data from the cloud, produces new authenticators for the corresponding new ciphertext, and then uploads the new ciphertext and the corresponding authenticators to the cloud. Obviously, this will consume a lot of computational and communicational resources of the users. Because none knows when the popularity of cloud data changes, it requires all the data owners being always online. Clearly, this naive solution is unpractical.

4.2. Slightly better solution

We consider to introduce a third party to reduce the user's computational and communicational resources. This party is in charge of generating data authenticators for users. Firstly, users should send their secret keys to the third party in advance. Once the popularity of cloud data changes, the third party will produce new authenticators for the data owners. Thus, data owners do not need to be always online and update their authenticators by themselves any longer. This solution is clearly better than the first solution. However, introducing the third party increases the complexity of the system and even causes some new security issues. Therefore, it is also not a good solution to the problem.

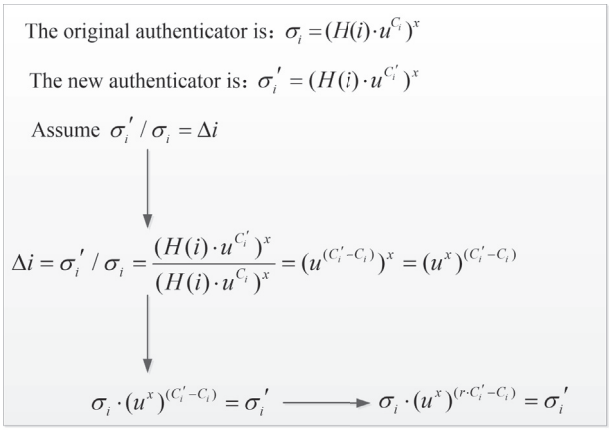


Fig. 2. The description of our idea.

4.3. Our core scheme

Our basic goal is to design a cloud storage auditing scheme with deduplication supporting different security levels according to data popularity. The unpopular data satisfy semantic security and the popular data can be deduplicated based on the ciphertext with lower security. The TPA can perform the integrity auditing of cloud data even if data popularity changes. In order to achieve our goal, we explore numerical relationship between old and new authenticators as shown in Fig. 2. In cloud storage auditing scheme, the homomorphic authenticator generated by user is $\sigma_i = (H(i)\cdot u^{C_i})^x$, where $H(\cdot)$ is a hash function, u is a generator of G_1, C_i is the i-th block of data ciphertext and x is the secret key of user. Assume the original authenticator is $\sigma_i = (H(i)\cdot u^{C_i})^x$ and the corresponding new authenticator is $\sigma_i' = (H(i)\cdot u^{C'_i})^x$. We set $\Delta i = \sigma_i'/\sigma_i = \frac{(H(i)\cdot u^{C_i'})^x}{(H(i)\cdot u^{C_i})^x} = (u^{(C_i'-C_i)})^x = (u^x)^{(C_i'-C_i)}$. Thus, $\sigma_i\cdot(u^x)^{(C_i'-C_i)} = \sigma_i'$. If one knows the original authenticator σ_i and $(u^x)^{(C_i'-C_i)}$, he can compute the corresponding new authenticator σ_i'. However, in this method, the adversary can calculate the inverse of $(C_i'-C_i)$ and forge valid authenticators for any ciphertext. For example, he can forge a valid authenticator $\sigma_i^* = \sigma_i\cdot(u^x)^{(C_i'-C_i)\cdot(C_i'-C_i)^{-1}\cdot(C_i^*-C_i)} = \sigma_i\cdot(u^x)^{(C_i^*-C_i)}$ for any ciphertext block C_i^*. For safety, we introduce a blind factor r. In our proposed scheme, the cloud on behalf of user produces new authenticators. User computes $(u^x)^{(r\cdot C_i'-C_i)}$ and uploads it to the cloud in advance. Because the cloud knows the original authenticator σ_i, he can produces the corresponding new authenticator σ' by computing $\sigma_i\cdot(u^x)^{(r\cdot C_i'-C_i)}$ when data popularity changes. Below, we give the detailed description of our core scheme.

- **Notations**: The file that the user wants to store in the cloud is denoted by F. Assume the file F is divided into n blocks $\{m_1, m_2, \ldots, m_n\}$ and each data block m_i has s sectors $\{m_{i1}, m_{i2}, \ldots, m_{is}\}$ where $1 \le i \le n$. We assume that a signature $SSig$ is used to ensure the integrity of the unique file identifier as the previous cloud storage auditing schemes (Wang et al., 2011, 2013). The secret key used to generate the signature $SSig$ is held by user, and the corresponding public key is published (see Table 2).
- **Description of our scheme** (see Fig. 3):
  1. ***Setup***: Input a security parameter κ. Then
     - Run $IG(1^\kappa)$ to generate two cyclic multiplicative groups of the same large prime order p denoted by G_1, G_2 respectively and a bilinear map $\hat{e}: G_1 \times G_1 \to G_2$.
     - Select three cryptographic hash functions $H_1: \{0,1\}^* \to G_1$, $H_2: G_2 \to \{0,1\}^l$, and $H_3: \{0,1\}^* \to G_1$. Choose a pseudo-random function (PRF) $\phi: Z_p^* \times Z_p^* \to Z_p^*$, a pseudo-random

Table 2
Notations.

Notations	Meaning
$\hat{e}$	A bilinear pairing map
H_1, H_2, H_3	Three different cryptographic hash functions
h	The index hash function
ϕ	A pseudo-random function (PRF)
π	A pseudo-random permutation (PRP)
ε	The symmetric cryptosystem
ε_C	The convergent encryption
ε_μ	The convergent threshold cryptosystem
C_ε	The ciphertext encrypted by symmetric cryptosystem
C_{ε_μ}	The ciphertext encrypted by convergent threshold cryptosystem
I_{ret}	An index
I_{FC}	An index for the ciphertext F_C
I_{rnd}	A random index which has the same length of I_{FC}
ctr	The number of different users who submit the same index
k_m	The encrypted key of convergent encryption
p	A large prime
G_1, G_2	Two multiplicative groups with order p
$g, \overline{g}$	Two generators of G_1 and G_2 respectively
Z_p^*	A prime field with nonzero elements
F	The data file shared in the cloud
m_i	The i-th block of the shared file, that is $F = \{m_1, m_2, \ldots, m_n\}$
n	The number of blocks in the shared file
s	The sector number of each file block
$\{u_1, u_2, \ldots, u_s\}$	The generators of G_1
k_{tag}	The secret key used to compute the authenticators
v	The verification key
x_i	A secret key share
T_i	The authenticator for data block m_i for $1 \le i \le n$
t	The popularity threshold
$chal$	The challenge message from the TPA
c	The number of challenged blocks
P	The Proof message from cloud
$name$	The file name

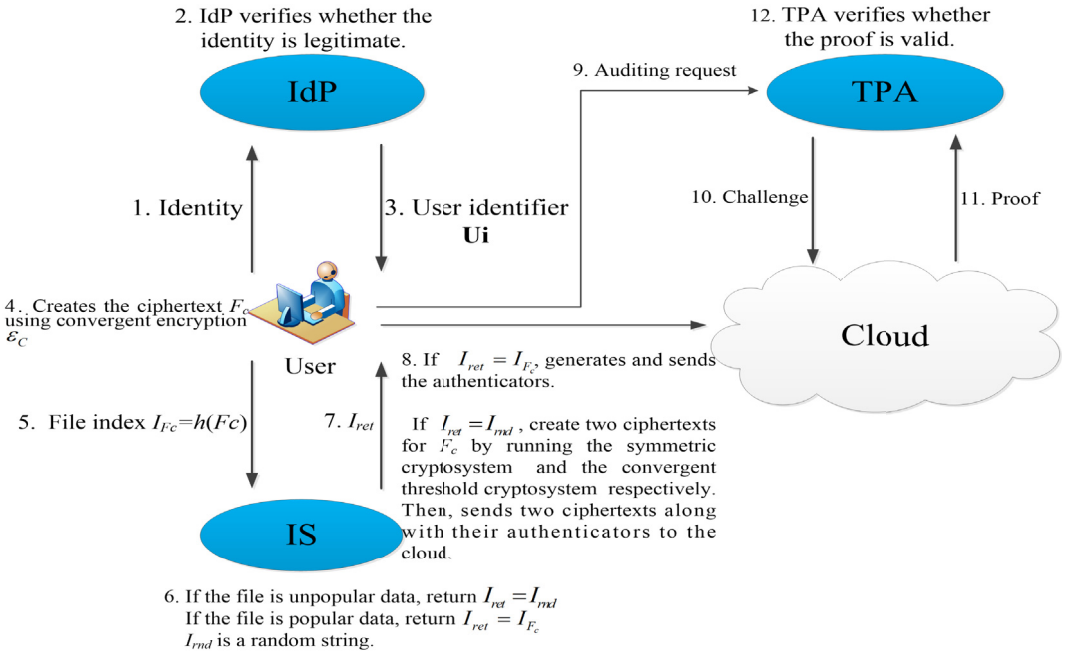


Fig. 3. The flow chart of the proposed scheme.

permutation (PRP) $\pi : Z_p^* \times \{1,2,\ldots,n\} \to \{1,2,\ldots,n\}$ and an indexing function $h : \{0,1\}^* \to \{0,1\}^*$.

– Run the algorithm $\varepsilon_\mu.Setup(\kappa, n, t) \to (pk, sk, S)$ to generate the system public key $pk = \{p, G_1, G_2, \hat{e}, H_1, H_2, H_3, h, g, g_{pub}\}$ and n key shares $\{x_i\}_{i=0}^{n-1}$.

2. ***Join***: Input the identity of the user U_i. When a user U_i wants to upload their files to the cloud, he firstly submits his identity to the identity provider (IdP). Then IdP verifies whether the $U_i's$ identity is legitimate. If it is, IdP issues a secret key share x_i and a user identifier **Ui** which is used to contact the cloud for such user. Otherwise, does not.
3. ***Upload***: Input the file F that the user wants to store in the cloud. Then

– The user **Ui** generates the ciphertext F_C using convergent encryption ε_C.
– In order to obtain an index I_{ret} that is used to interact with cloud subsequently, the user **Ui** communicates with the indexing service (IS). Firstly, the user executes the indexing function h to produce an index I_{F_C} for the ciphertext F_C. Then the user submits this index I_{F_C} to the indexing service (IS). IS receives this index (a set denoted by *index*) and keeps the number of users (denoted by *ctr*) who submit the same index. If this number is less than the popularity threshold t, IS invokes a PRF on the index I_{F_C} and the user's identity with a random seed σ to generate a bitstring I_{rnd}. The length of I_{rnd} is the same as the index I_{F_C}. Then, IS replies with $I_{ret} = I_{rnd}$. Otherwise, IS replies with $I_{ret} = I_{Fc}$. The detailed processes of getting the index I_{ret} is shown in Fig. 4. According to IS's reply, the user **Ui**

Ui: $I_{F_c} \leftarrow h(F_c)$

Ui: ⟶ IS: I_{Fc}

IS: if $ctr > t$

return $I_{ret} = I_{Fc}$;

$I_{rnd} = PRF_\sigma(U_i \| I_{F_C})$

if $I_{rnd} \notin index$

increment ctr;

add I_{rnd} to the set $index$;

if $ctr = t$

return I_{rnd}, and the set $index$;

else

return $I_{ret} = I_{rnd}$;

Fig. 4. The detail process of getting the index I_{ret}.

Ui: $k_m \leftarrow \varepsilon_C.K(F)$

$F_C \leftarrow \varepsilon_C.E(k_m, F)$

$I_{F_C} \leftarrow h(F_C)$

Ui ⟶ IS: I_{Fc}

IS ⟶ **Ui**: return I_{ret};

if $I_{ret} = I_{Fc}$

execute *Upload.Popular*;

else if

the IS return the set *index*;

execute *Upload.Unpopular*;

execute *Upload.Reclaim*;

else

execute *Upload.Unpopular*;

Fig. 5. The detailed processes of upload operation.

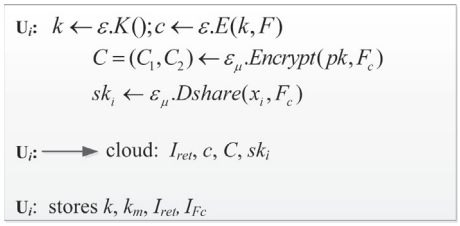


Fig. 6. The *Upload.Unpopular* algorithm.

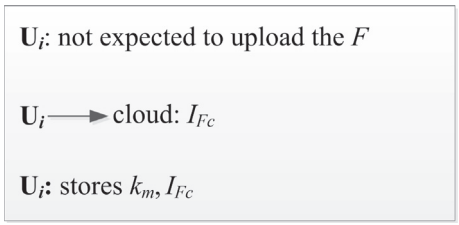


Fig. 7. The *Upload.Popular algorithm.*

determines which one of the following sub-algorithms is performed. We give the detailed description of the upload operation in Fig. 5.

* ***Upload.Unpopular***(Fig. 6): If the response I_{ret} from the IS equals to a random index I_{rnd}, then the file F is still unpopular now. The number of users who submit the same index is less than the popularity threshold t. The user needs to create two ciphertexts C_ε and C_{ε_μ} for F_C by running the symmetric cryptosystem ε and the convergent threshold cryptosystem ε_μ respectively. Then he uploads both of them to the cloud. If this file has not become popular, the user Ui can recover this file from the ciphertext C_ε. If this file has become popular, the cloud can recover the encryption key of the convergent threshold cryptosystem ε_μ and perform deduplication. Finally, the user **Ui** deletes the file F and holds two file index I_{ret}, I_{F_C}, and two encryption key k and k_m for ε, ε_c respectively.
* ***Upload.Popular***(Fig. 7): If the response I_{ret} from the IS equals to the index I_{F_C} for the ciphertext F_C, then the file F has already become popular. The number of users who submit the same index is no less than the popularity threshold t. The user **Ui** does not need to upload this file any more. He deletes the file F and holds the index I_{F_C} and the encryption k_m locally.

4. ***AuthGen***: Input a secret key $k_{tag} \leftarrow Z_p^*$ and a ciphertext of file $C = \{c_1, c_2, \ldots, c_n\}$ (specially C is C_ε or C_{ε_μ}). The user computes and publishes the key $v \leftarrow g^{k_{tag}}$. **Ui** generates the authenticator T_i for each ciphertext block c_i $(1 \le i \le n)$ and uploads these authenticators to the cloud.
   - **Ui** chooses s generators $u_1, u_2, \ldots, u_s$ of G_1, and random selects $r \leftarrow Z_p^*$.
   - Let τ_0 be $name \parallel n \parallel v^r \parallel u_1 \parallel u_2 \parallel \ldots \parallel u_s$. The user randomly selects a signing key $ssk \leftarrow Z_p^*$ and computes the corresponding verification key $P_{ssk} \leftarrow g^{ssk}$. The file tag is τ_0 together with a signature on τ_0 under the secret key ssk: $\tau \leftarrow \tau_0 \parallel SSig_{ssk}(\tau_0)$.
   - **Ui** computes authenticator for each data block:

$$T_i = (H_3(name\|i) \cdot \prod_{j=1}^{s} u_j^{c_{ij}})^{k_{tag}}$$

where c_{ij} denotes the encrypted j-th sector of the i-th data block.

   - **Ui** computes $\{u_1^{k_{tag}(r(C_{\varepsilon_\mu})_{i,1}-(C_\varepsilon)_{i,1})}, u_2^{k_{tag}(r(C_{\varepsilon_\mu})_{i,2}-(C_\varepsilon)_{i,2})}, \ldots, u_s^{k_{tag}(r(C_{\varepsilon_\mu})_{i,s}-(C_\varepsilon)_{i,s})}\}$ and sends it to IS.
   - **Ui** sends $\{T_i\}_{1\le i\le n}$ and the file tag to the cloud.
5. ***PopulartityChange***:(Fig. 8): This algorithm is executed when the number of users submitting the same index is up to the popularity threshold t. The user **Ui** does not need to upload the file F to cloud again. **Ui** sends the set *index* received from the IS to the cloud. According to this index set, the cloud can collect decryption shares of different users whose file index is included in the set *index*. After that, the cloud can decrypt the ciphertext C_{ε_μ} uploaded by these users respectively. Then the cloud obtains the ciphertext F_C encrypted by the convergent encryption. The deduplication can be achieved as the ciphertext F_C is the same for file F. Finally, the IS sends $\{u_1^{k_{tag}(r(C_{\varepsilon_\mu})_{i,1}-(C_\varepsilon)_{i,1})}, u_2^{k_{tag}(r(C_{\varepsilon_\mu})_{i,2}-(C_\varepsilon)_{i,2})}, \ldots, u_s^{k_{tag}(r(C_{\varepsilon_\mu})_{i,s}-(C_\varepsilon)_{i,s})}\}$ to the cloud. The cloud creates the new data block authenticator $T_i' = T_i \cdot \prod_{j=1}^{s} u_j^{k_{tag}\cdot(rc_{\varepsilon\mu_{ij}}-c_{\varepsilon_{ij}})}$ for each user.
6. ***ProofGen***: Input the data blocks $\{c_i\}_{1\le i\le n}$ and the corresponding authenticators $\{T_i\}_{1\le i\le n}$.
   - TPA generates the auditing challenge including the following phases:
     * The TPA gains the file tag from the cloud and verifies whether the signature on τ_0 is valid or not using the key *Pssk*. If the signature is invalid, TPA rejects and halts.
     * Otherwise, the TPA recovers file name *name*, n, v^r and $\{u_1, u_2, \ldots, u_s\}$. Then he chooses a number $c(1 \le c \le n)$ as the number of the challenged blocks.
     * The TPA picks two random numbers $k_1 \leftarrow Z_p^*, k_2 \leftarrow Z_p^*$.
     * The TPA sends the challenge $chal = (c, k_1, k_2)$ to the cloud.
   - After receiving the challenge message *chal* from the TPA, the cloud computes $l_t = \pi_{k_1}(t)$ and $a_t = \phi_{k_2}(t)$ for $1 \le t \le c$. And then he computes $T = \prod_{t=1}^{c} T_{l_t}^{a_t}$, $\eta_j = \sum_{t=1}^{c} a_t \cdot c_{l_t,j}, 1 \le j \le s$.
7. ***ProofVerify***: Input the challenge massage $chal = (c, k_1, k_2)$ and the Proof $P = (T, \eta)$. After receiving the proof P from the cloud, TPA computes $l_t = \pi_{k_1}(t)$ and $a_t = \phi_{k_2}(t)$ for $1 \le t \le c$. Then he checks whether the following verification equation holds or not.

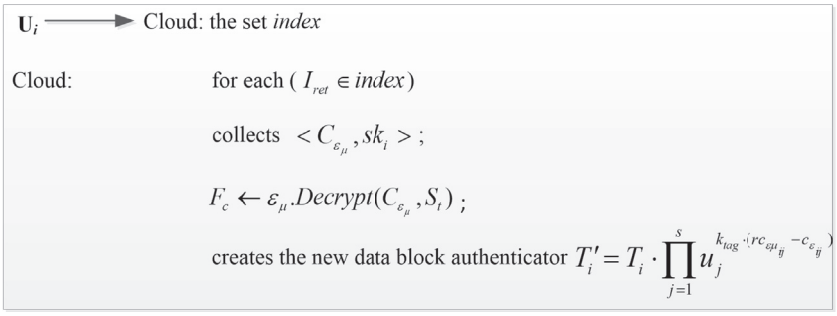


Fig. 8. The *PopulartityChange* algorithm.

$$\hat{e}(T,g) \stackrel{?}{=} \hat{e}(\prod_{t=1}^{c}(H_3(name\|l_t)^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta_j}), v) \quad (1)$$

$$\hat{e}(T,g) \stackrel{?}{=} \hat{e}(\prod_{t=1}^{c}(H_3(name\|l_t)^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta_j}), v^r) \quad (2)$$

If equation (1) or (2) holds, it means that the cloud stores the target file completely. Otherwise, does not. Finally, TPA returns this result to user. The workflow of our scheme shows in Fig. 9.

the correctness: For one challenge $chal = (c, k_1, k_2)$ and a valid Proof P, the *ProofVerify* algorithm always returns *True* according to equation (1). It is because the verification equation (1) holds.

$$\begin{aligned} e(T,g) &= \hat{e}(\prod_{t=1}^{c} T_{l_t}^{a_t}, g) \\ &= \hat{e}(\prod_{t=1}^{c}((H_3(name\|l_t) \cdot \prod_{j=1}^{s} u_j^{c_{l_t j}})^{k_{tag}})^{a_t}, g) \\ &= \hat{e}(\prod_{t=1}^{c}((H_3(name\|l_t))^{a_t} \cdot (\prod_{j=1}^{s} u_j^{c_{l_t j}})^{a_t})^{k_{tag}}, g) \\ &= \hat{e}(\prod_{t=1}^{c}((H_3(name\|l_t))^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta_j}, g^{k_{tag}}) \\ &= \hat{e}(\prod_{t=1}^{c}((H_3(name\|l_t))^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta_j}, v) \end{aligned}$$

Similarly, the correctness of equation (2) can also be proved.

5. Security and performance

5.1. Security analysis

In this part, we prove that the security of the proposed scheme from the following segments: the privacy of user's secret key, the auditing soundness, and the detectability.

Theorem 1. (*the privacy of user's secret key*) The cloud cannot know any useful information about the user's secret key though he can generate the new authenticator for each block on behalf of user.

Proof. In order to release user's computation and communication burden, the cloud instead of user produces new authenticators in this scheme. The user computes $\{u_1^{k_{tag}}, u_2^{k_{tag}}, \ldots, u_s^{k_{tag}}\}$ and uploads it along with the file F and authenticators to the cloud. As long as the discrete logarithm (DL) problem in G_1 is hard, the cloud cannot

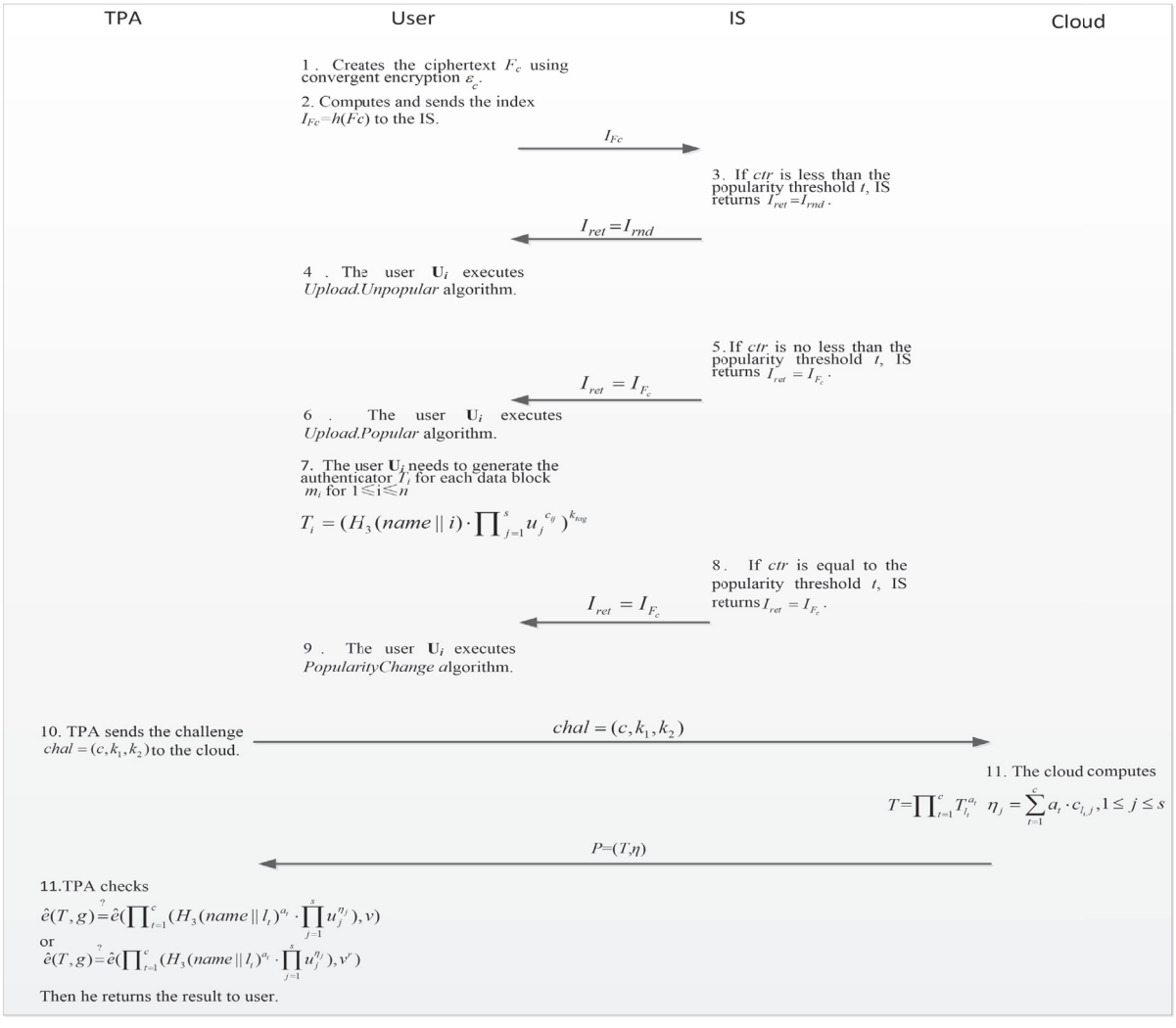


Fig. 9. The workflow of our scheme.

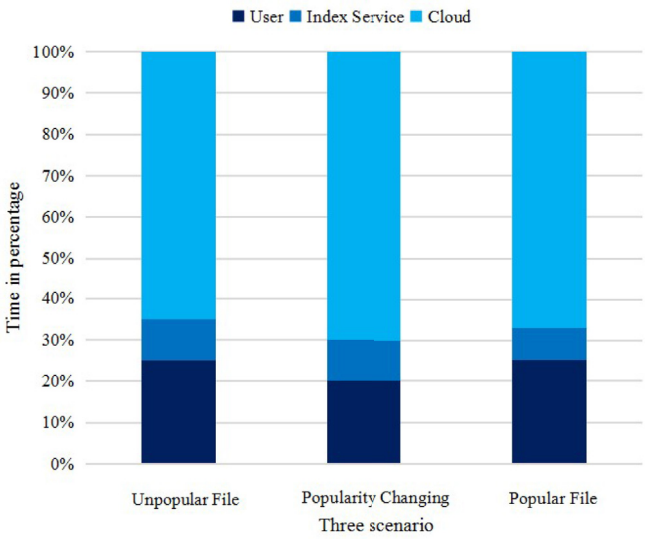


Fig. 10. The computation time percentage in different scenarios.

obtain the user's secret key k_{tag} from these $\{u_1{}^{k_{tag}}, u_2{}^{k_{tag}}, \ldots, u_s{}^{k_{tag}}\}$. Because the cloud knows the original authenticator T_i, he can produce the corresponding new authenticator T_i' by computing $T_i' = T_i \cdot \prod_{j=1}^{s} u_j^{k_{tag}\cdot(m'_{ij}-m_{ij})}$ when data popularity changes. Therefore, the cloud cannot know any information about the user's secret key though he can generate the new authenticator for each block on behalf of user.

Theorem 2. (*the auditing soundness*) The *ProofVerify* algorithm returns "true" if and only if the cloud stores the user data completely.

Proof. We will prove this theorem using a line of games.

Game 0. It is as good as the Game 0 in (Bellare and Goldreich, 1992).

Game 1. The Game 1 has just one difference from the Game 0. In Game 1, the challenger keeps his answers for all of adversary's queries. If a tag τ passes the verify, but it is not in the answers stored by the challenger, the adversary wins Game 1.

Analysis. If the advantage of the adversary wins Game 1 is non-negligible, the *SSig* signature scheme is not security. It contradicts with the definition of the *SSig* signature scheme. Analogously, it means *name*, t and $\{u_1, u_2, \ldots, u_s\}$ are all produced by the challenger.

Game 2. The Game 2 has just one difference from the Game 1. The challenger stores his answers for the adversary's queries as Game 1. If the valid aggregate authenticator T′ does not equal to $\prod_{t=1}^{c} T_{l_t}^{a_t}$, then the adversary succeeds.

Analysis. Assume that the valid Proof produced by the honest prover is (T, η), where $\eta = \{\eta_1, \ldots, \eta_s\}$. According to the correctness of the scheme, this verification equation holds:

$$\hat{e}(T,g) \stackrel{?}{=} \hat{e}\Big(\prod_{t=1}^{c}(H_3(name\|l_t)^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta_j}), v\Big) \tag{3}$$

Assume that the response (T', η') (where $\eta' = \{\eta_1', \eta_2', \ldots, \eta_s'\}$) is not produced by the challenger, but it passes the verify. Thus, the following verification equation holds:

$$\hat{e}(T',g) = \hat{e}\Big(\prod_{t=1}^{c}(H_3(name\|l_t)^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta'_j}), v\Big) \tag{4}$$

The verification equation holds in two cases. One is $T' = T$, where $\eta_j' = \eta_j$ for each j. Another is the $T' \neq T$. Namely, at least one of $\Delta\eta_j \neq 0$, where $\Delta\eta_j = \eta_j' - \eta_j$ for $1 \leq j \leq s$. If the adversary wins this game, we can design a simulator to break through the CDH problem.

Given $(g, g^{k_{tag}}, h) \in G_1$, the simulator targets to output $h^{k_{tag}}$. He selects two random values $\alpha, \beta \leftarrow Z_p^*$, and computes $u = g^{\alpha}h^{\beta}$. And the verification key $v = g^{k_{tag}}$.

For each j, $1 \leq j \leq s$, the simulator chooses random values $\alpha_j, \beta_j \leftarrow Z_p^*$ and sets $u_j = g^{\alpha_j}h^{\beta_j}$. The simulator selects a random value $r_i \leftarrow Z_p^*$, for each i ($1 \leq i \leq n$). Then, he constructs the random oracle at i as:

$$H_3(name\|l_t) = g^{r_i}/(g^{\sum_{j=1}^{s}\alpha_j m_{ij}} \cdot h^{\sum_{j=1}^{s}\beta_j m_{ij}}) \tag{5}$$

We have

$$\begin{aligned} H_3(name\|l_t) \cdot \prod_{j=1}^{s} u_j^{m_{ij}} &= g^{r_i}/(g^{\sum_{j=1}^{s}\alpha_j m_{ij}} \cdot h^{\sum_{j=1}^{s}\beta_j m_{ij}}) \cdot \prod_{j=1}^{s} u_j^{m_{ij}} \\ &= g^{\sum_{j=1}^{s}\alpha_j m_{ij}} \cdot h^{\sum_{j=1}^{s}\beta_j m_{ij}} \cdot g^{r_i}/(g^{\sum_{j=1}^{s}\alpha_j m_{ij}} \cdot h^{\sum_{j=1}^{s}\beta_j m_{ij}}) \\ &= g^{r_i} \end{aligned}$$

The simulator calculates $T_i' = (H_3(name\|l_t) \cdot \prod_{j=1}^{s} u_j^{m'_{ij}})^{k_{tag}} = (g^{r_i})^{k_{tag}} = (g^{k_{tag}})^{r_i}$. From results of dividing equation (5) by equation (4), we get

$$\hat{e}(T'/T, g) = \hat{e}(\prod_{j=1}^{s} u_j^{\Delta\eta_j}, v) = \hat{e}(\prod_{j=1}^{s}(g^{\alpha_j}h^{\beta_j}), v).$$

We obtain

$$\hat{e}(T' \cdot T^{-1} \cdot v^{-\sum_{j=1}^{s}\alpha_j\cdot\Delta\eta_j}, g) = e(h, v)^{\sum_{j=1}^{s}\beta_j\cdot\Delta\eta_j} \tag{6}$$

So

$$h^{k_{tag}} = (T' \cdot T^{-1} \cdot v^{-\sum_{j=1}^{s}\beta_j\Delta\eta_j})^{1/(\sum_{j=1}^{s}\beta_j\Delta\eta_j)}$$

When $\beta \cdot \Delta\eta_j \equiv 0 \bmod p$, whose probability is negligible, the game fails. Thus, the CDH problem can be solved by this simulator.

Game 3. The Game 3 has just one difference from the Game 2. In Game 3, the challenger still stores the answers of adversary's queries. For one of these instances, if the aggregate message η_j' submitted by the adversary does not equal to $\sum_{t=1}^{c} a_t \cdot m_{l_t j}$, but it passes the verify. Then the challenger aborts.

Analysis. Suppose the valid Proof produced by the honest prover is (T, η), where $\eta = \{\eta_1, \eta_2, \ldots, \eta_s\}$. According to the correctness of the scheme, the equation $\hat{e}(T,g) = \hat{e}(\prod_{t=1}^{c}(H_3(name\|l_t)^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta_j}), v)$ holds. Assume that the response (T', η') (where $\eta' = \{\eta_1', \eta_2', \ldots, \eta_s'\}$) generated by the adversary is not equal to an honest prover generated. Because (T', η') passes the verify, so $\hat{e}(T',g) \stackrel{?}{=} \hat{e}(\prod_{t=1}^{c}(H_3(name\|l_t)^{a_t} \cdot \prod_{j=1}^{s} u_j^{\eta'_j}), v)$ holds. Note that Game 2 already guaranteed $T' = T$. Define $\Delta\eta_j = \eta_j' - \eta_j$ again. From Game 2, we know at least one of $\Delta\eta_j \neq 0$. If the adversary wins Game 3, there is a simulator to break through the DL problem. Given $(g, h) \in G_1$, the simulator targets to output x, which satisfies $h = g^x$. For each $1 \leq j \leq s$, the simulator selects random values $\alpha_j, \beta_j \in Z_p^*$ and sets $u_j = g^{\alpha_j} \cdot h^{\beta_j}$.

According to the above two verification equations, we get

$$\begin{aligned} e(\prod_{i=1}^{c} H_3(name\|l_t)^{v_i} \cdot \prod_{j=1}^{s} u_j^{\eta_j}, g^{k_{tag}}) &= \hat{e}(T,g) \\ &= \hat{e}(T',g) \\ &= \hat{e}(\prod_{i=1}^{c} H_3(name\|l_t)^{v_i} \cdot \prod_{j=1}^{s} u_j^{\eta'_j}, g^{k_{tag}}) \end{aligned}$$

So

$$\prod_{j=1}^{s} u_j^{\eta_j} = \prod_{j=1}^{s} u_j^{\eta'_j}$$

We know

$$1 = \prod_{j=1}^{s} u_j^{\Delta\eta_j} = \prod_{j=1}^{s}(g^{\alpha_j} \cdot h^{\beta_j})^{\Delta\eta_j} = g^{\sum_{j=1}^{s}\alpha_j\Delta\eta_j} \cdot h^{\sum_{j=1}^{s}\beta_j\Delta\eta_j}$$

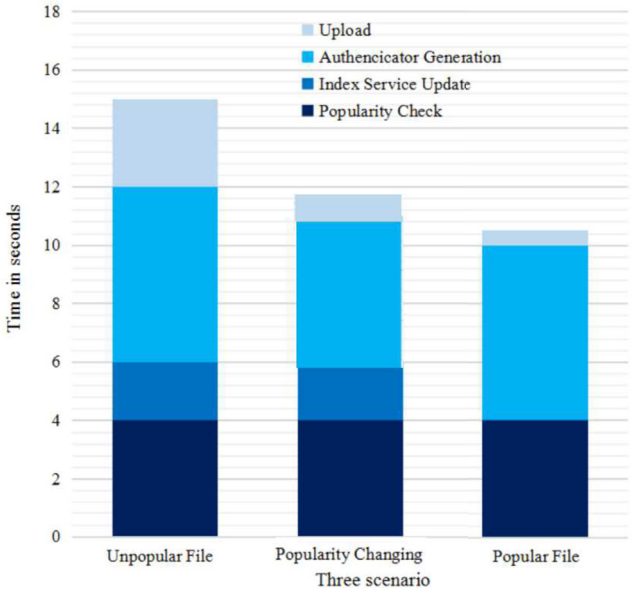


Fig. 11. The total computation time for uploading a file in different scenarios.

If there exists a $\Delta\eta_j \neq 0$, the DL problem can be solved:

$$h = g^{\frac{-\sum_{j=1}^{s}\alpha_j\cdot\Delta\eta_j}{\sum_{j=1}^{s}\beta_j\cdot\Delta\eta_j}} \Rightarrow x = -\sum_{j=1}^{s}\alpha_j / \sum_{j=1}^{s}\beta_j$$

The probability that the denominator is zero can be ignored. Thus, this simulator can break through the DL problem with high probability. Finally, we can prove that the differences between these games can be ignored.

The way to design a knowledge extractor to extract all of challenged file blocks $m_1, \ldots, m_c$ is to get c independent linear equations with the variables $m_1, \ldots, m_c$. In order to establish these linear equations, we firstly select coefficients $v_1, \ldots, v_c$, which mutual independent with each other.

Theorem 3. (*the detectability*): Assume the file stored in the cloud is divided into n blocks, and these blocks contain b bad (deleted or modified) blocks and c challenged blocks. The proposed cloud storage auditing scheme is $(\frac{b}{n}, 1-(\frac{n-1}{n})^c)$ detectable.

Proof. Assume that the file stored in the cloud is divided into n blocks, and these blocks contain m bad (deleted or modified) blocks and c challenged blocks. If the verifier chooses one or more bad blocks as the challenged blocks, then bad blocks can be detected. Let X be a variable denoting the number of bad blocks challenged by the challenger. Let PX denote the probability that there is at least one above blocks. We have

$$\begin{aligned} P_X &= P\{X \geq 1\} \\ &= 1 - P\{X = 0\} \\ &= 1 - \frac{n-b}{n}\frac{n-1-b}{n-1} \times \cdots \times \frac{n-c+1-b}{n-c+1} \end{aligned}$$

We know $P_X \geq 1-(\frac{n-b}{n})^c$. Thus, this Theorem holds.

5.2. Performance analysis

We carry out a series of simulation experiments to assess the performance of our scheme. The experiments carries out on a Linux server that used Intel's 2.70 GHz processor and 4 GB of RAM. Using the Pairing-Based Cryptography (PBC) library (Wang et al., 2016) and the GNU Multiple Precision Arithmetic (GMP), we realize the simulation experiment using the C language. We set the base field size to be 512 bits, the size of an element in Z_p^* to be 160 bits, and the outsourced file to be 20 MB consisting of 1000 blocks .

From Fig. 10, we can see that the computation time from user occupies about 25% of the total computation time and the computation time from cloud occupies about 65% in the unpopular file scenario. When data popularity changes, the computation time from user occupies only about 20%. In the popular file scenario, the computation time from

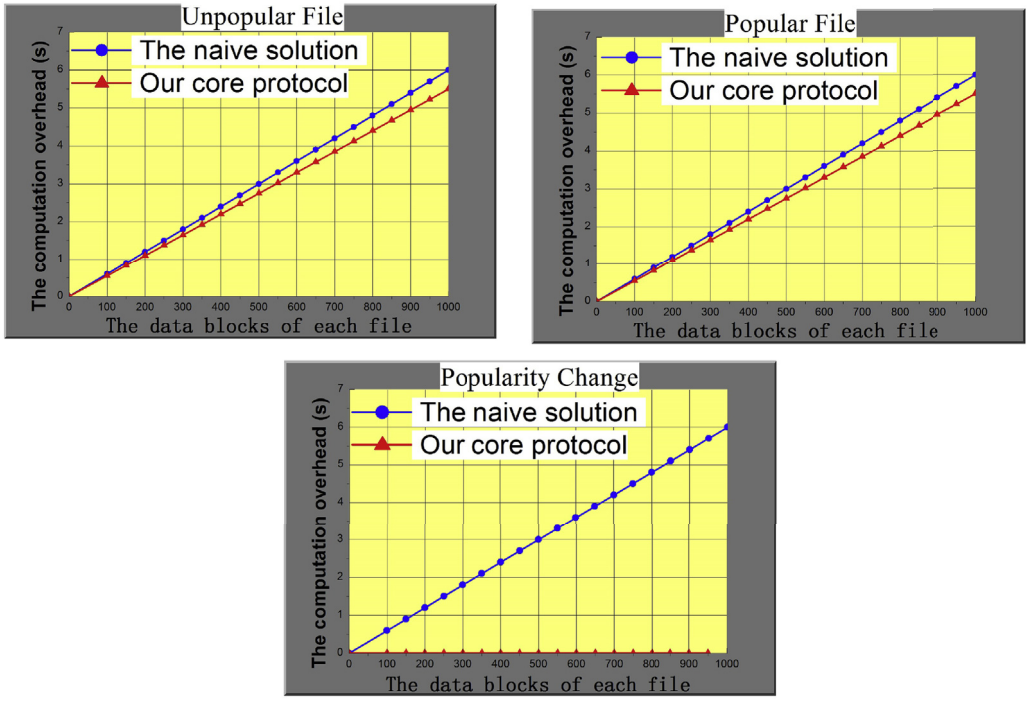


Fig. 12. The computation time of authenticator generation in different scenarios.

user is similar to that in the unpopular scenario. The computation time from user occupies about 25% of the total computation time and the computation time from cloud occupies about 65%. That is, the computation and communication cost on the user side is very similar when the data is popular or unpopular. It can be concluded that even if we divide the data into popular data and unpopular data, the computation and communication cost on the user side does not increase. In general, the computation cost on the user side in our deduplication scheme is reasonable.

In Fig. 11, we show the total computation time for uploading a file in different scenarios. In the unpopular file scenario, the time of the popularity checking is about 4s, the time of the index service update is about 2s, the time of the authenticator generation is about 6s, and the time of the file uploading is about 3s. In the popularity changing scenario, the time of the authenticator generation is 0s. The reason is that, in our scheme, the cloud computes the new authenticators for the new ciphertext on behalf of user in popularity changing scenario. The time from file uploading is much less than that in the unpopular file scenario. The reason is that the user does not need to upload the file any more when the uploaded file becomes popular. From Fig. 11, we can see that the time from index service update is 0s in the popular file scenario and the time from file uploading is much less than that in the popularity changing scenario. The reason is that the user does not need to upload the file any more when the uploaded file is the popular file.

From Fig. 12, we can see that the computation time of authenticator generation in unpopular file scenario is as same as that in popular file scenario. The computation time of authenticator generation in our core scheme is clearly less than that in the naive solution. In the naive solution, each owner downloads all his previously stored data from the cloud, produces new authenticators for the corresponding new ciphertext, and then uploads the new ciphertext and the corresponding authenticators to the cloud. However, in our core scheme, users do not need to download the whole data, because the cloud can generate the new authenticators instead of them. So, the user does not do any extra operation, the computation time of authenticator generation is 0s in popularity changing scenario. This time in naive solution ranges from 0s to the 5.98s with the corresponding number of file blocks ranging from 0 to 1000.

The audit process can be divided into three sub-processes. The first is challenge process, the TPA sends the challenge messages to the cloud. The second is Proof generation process, the cloud generate a proof after receiving the challenge messages. The final is verification process, the TPA checks whether the proof is correct or not. From Fig. 13, The most time-consuming process is the verification process, ranging from 0.216s to 2.049s. The least time consuming is the challenge phase, ranging from 0.026s to 0.263s. Finally, the time of the proof generation ranges from 0.178s to 1.785s. And we can see that the time of authentication, challenge, and proof generation is linear with the number of the challenged blocks. With the number of challenged blocks increasing, the time increases linearly.

As shown in Table 3, each owner downloads his previously stored data from the cloud, produces new authenticators for the corresponding new ciphertext, and then uploads the new ciphertext and the corresponding authenticators to the cloud in naive solution. In the proposed scheme, users do not need to download the whole data because the cloud can generate the new authenticators instead of them. Users also do not need to be always online in the proposed scheme. Obviously, these operations in the naive solution will consume a lot of computation and communication resources of the users. Because none knows when the popularity of cloud data changes, it requires data owners being always online. Clearly, this naive solution is unpractical.

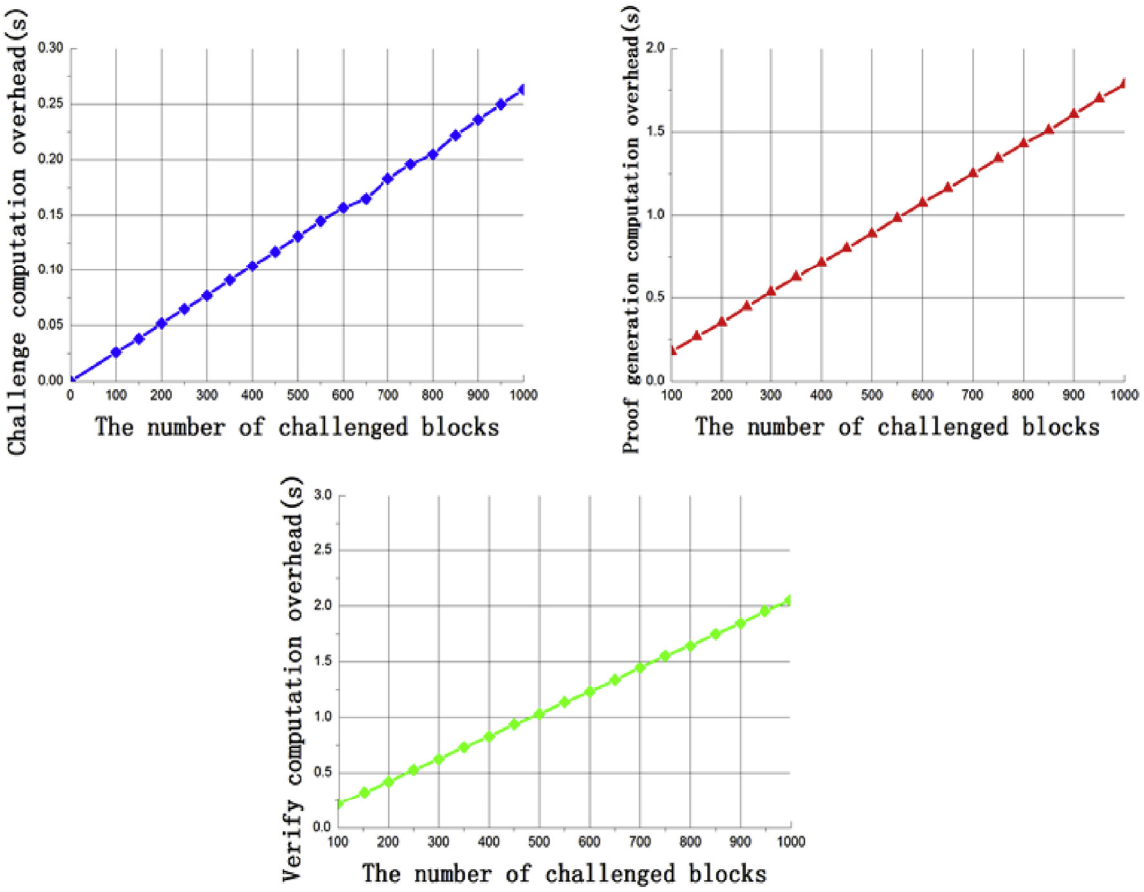


Fig. 13. Computation overheads of challenge, Proof generation and verification.

Table 3
The comparison between the naive solution and the proposed scheme.

Aspects	Naive solution	The proposed scheme
All users download the whole their cloud data	✓	×
Users generate new authenticator	✓	×
Users need to be always online	✓	×
The efficiency	low	high

6. Conclusion

In this paper, we explore how to realize cloud storage auditing with deduplication supporting different security levels according to data popularity. In order to achieve our goal, we overcome one main challenge to ensure that the TPA still can audit the integrity of cloud data after data popularity changes. Then, we propose the first cloud storage auditing scheme with deduplication supporting different security levels. In our scheme, we adopt convergent threshold cryptosystem to provide semantic security for unpopular data and ciphertext deduplication for popular data. Users do not need to produce new authenticators by himself and be online when the data popularity changes. The experimental results demonstrate that our scheme is efficient.

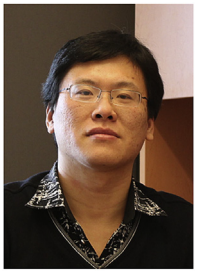
Acknowledgment

This research is supported by National Natural Science Foundation of China (61572267, 61602275), National Cryptography Development Fund of China (MMJJ20170118), the Open Project of Co-Innovation Center for Information Supply and Assurance Technology, Anhui University, Jiangsu Key Laboratory of Big Data Security and Intelligent Processing, NJUPT(BDSIP1806), the Open Project of the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences.

References

Alkhojandi, N., Miri, A., 2015. Privacy-preserving public auditing in cloud computing with data deduplication. In: 7th International Symposium on Foundations and Practice of Security, pp. 35–48.
Ateniese, G., Burns, R., Curtmola, R., Herring, J., Kissner, L., Peterson, Z., Song, D., 2007. Provable Data Possession at Untrusted Stored, pp. 598–609.
Ateniese, G., Pietro, R., Mancini, L., Tsudik, G., 2008. Scalable and Efficient Provable Data Possession, pp. 1–10.
Aujla, G., Chaudhary, R., Kumar, N., Das, A., Rodrigues, J., 2018. SecSVA: secure storage, verification, and auditing of big data in cloud environment. IEEE Commun. Mag. 56 (1), 78–85.
Bellare, M., Goldreich, O., 1992. On defining proofs of knowledge. In: 12th Annual International Cryptology Conference, pp. 390–420.
Douceur, J., 2002. The sybil attack. In: International Workshop on Peer-To-Peer Systems, pp. 251–260.
Gantz, J., Reinsel, D., 2010. The Digital Universe Decade Are You Ready? http://www.emc.com/collateral/analyst-reports/idc-digital-universe-are-you-ready.pdf.
Halevi, S., Harnik, D., Pinkas, B., Shulman-Peleg, A., 2011. Proofs of ownership in remote storage systems. In: 18th ACM Conference on Computer and Communications Security, pp. 491–500.
Hou, H., Yu, J., Zhang, H., Xu, Y., Hao, R., 2018. Enabling secure auditing and deduplicating data without owner-relationship exposure in cloud storage. Clust. Comput., https://doi.org/10.1007/s10586-018-2813-8.
Li, J., Li, J., Xie, D., 2016. Secure auditing and deduplicating data in cloud. IEEE Trans. Comput. 65 (8), 2386–2396.
Liu, C., Ranjan, R., Zhang, X., Yang, C., Georgakopoulos, D., Chen, J., 2013. Public auditing for big data storage in cloud computing-A survey. In: 16th IEEE International Computational Science and Engineering, pp. 1128–1135.
Pietro, R., Sorniotti, A., 2012. Boosting efficiency and security in proof of ownership for deduplication. In: 7th ACM Symposium on Information, Computer and Communications Security, pp. 81–82.
Shacham, H., Waters, B., 2008. Compact proofs of retrievability. In: 14th International Conference on the Theory and Application of Cryptology and Information Security, pp. 90–107.
Shen, J., Shen, J., Chen, X., Huang, X., Susilo, W., 2017. An efficient public auditing scheme with novel dynamic structure for cloud data. IEEE Trans. Inf. Forensics Secur. 12 (10), 2402–2415.
Shen, W., Yu, J., Xia, H., Zhang, H., Lu, X., Hao, R., 2017. Light-weight and privacy-preserving secure cloud auditing scheme for group users via the third party medium. J. Netw. Comput. Appl. 82, 56–64.
Shen, W., Qin, J., Yu, J., Hao, R., 2019. Identity-based integrity auditing and data sharing with sensitive information hiding for secure cloud storage. IEEE Trans. Inf. Forensics Secur. 14, 331–346.
Stanek, J., Sorniotti, A., Androulaki, E., Kencl, L., 2014. A secure data deduplication scheme for cloud storage. In: International Conference on Financial Cryptography and Data Security, pp. 99–118.
Wang, C., Wang, Q., Ren, K., Lou, W., 2010. Privacy-preserving public auditing for data storage security in cloud computing. In: IEEE Conference on Computer Communications, pp. 525–533.
Wang, Q., Wang, C., Ren, K., Lou, W., Li, J., 2011. Enabling public auditability and data dynamics for storage security in cloud computing. IEEE Trans. Parallel Distrib. Syst. 22 (5), 847–859.
Wang, C., Chow, S., Wang, Q., Ren, K., Lou, W., 2013. Privacy preserving public auditing for secure cloud storage. IEEE Trans. Comput. 62 (2), 362–375.
Wang, H., He, D., Tang, S., 2016. Identity-based proxy-oriented data uploading and remote data integrity checking in public cloud. IEEE Trans. Inf. Forensics Secur. 14 (6), 1165–1176.
Wazid, M., Das, A., Hussain, R., Succi, G., Rodrigues, J., 2018. Authentication in cloud-driven IoT-based big data environment: survey and outlook. J. Syst. Architect., https://doi.org/10.1016/j.sysarc.2018.12.005.
Yang, G., Yu, J., Shen, W., Su, Q., Zhang, F., Hao, R., 2016. Enabling public auditing for shared data in cloud storage supporting identity privacy and traceability. J. Syst. Software 113, 130–139.
Yu, J., Wang, H., 2017. Strong key-exposure resilient auditing for secure cloud storage. IEEE Trans. Inf. Forensics Secur. 12 (8), 1931–1940.
Yu, J., Ren, K., Wang, C., Varadharajan, V., 2015. Enabling cloud storage auditing with key-exposure resistance. IEEE Trans. Inf. Forensics Secur. 10 (6), 1167–1179.
Yu, J., Ren, K., Wang, C., 2016. Enabling cloud storage auditing with verifiable outsourcing of key updates. IEEE Trans. Inf. Forensics Secur. 11 (6), 1362–1375.
Yuan, J., Yu, S., 2013. Secure and constant cost public cloud storage auditing with deduplication. In: IEEE Conference on Communications and Network Security, pp. 145–153.
Yuan, J., Yu, S., 2014. Efficient public integrity checking for cloud data sharing with multi-user modification. In: IEEE Conference on Computer Communications, pp. 2121–2129.
Zhang, Y., Zhang, H., Yu, J., Hao, R., 2018. Authorized identity-based public cloud storage auditing scheme with hierarchical structure for large-scale user groups. China Commun. 15 (11), 111–121.
Zhang, Y., Yu, J., Hao, R., Wang, C., Ren, Kui, 2018. Enabling efficient user revocation in identity-based cloud storage auditing for shared big data. IEEE Trans. Dependable Secure Comput., https://doi.org/10.1109/TDSC.2018.2829880.
Zheng, Q., Xu, S., 2012. Secure and efficient Proof of storage with deduplication. In: ACM Conference on Data and Application Security and Privacy, pp. 1–12.

Huiying Hou received the B.S. and M.S. degrees from the College of Computer Science and Technology, Qingdao University, China, in 2015 and 2018, respectively. She is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Fudan University, China. Her research interests include cloud security and cryptography.



Jia Yu is a professor of the College of Computer Science and Technology and the department director of Information Security at Qingdao University. He received the M.S. and B.S. degrees in School of Computer Science and Technology from Shandong University in 2003 and 2000, respectively. He received Ph. D. degree in Institute of Network Security from Shandong University, in 2006. He was a visiting professor with the Department of Computer Science and Engineering, the State University of New York at Buffalo, from Nov. 2013 to Nov. 2014. His research interests include cloud computing security, key evolving cryptography, digital signature, and network security. He has published over 100 academic papers. He is the reviewer of more than 40 international academic journals.

Rong Hao is currently with the College of Computer Science and Technology, Qingdao University. Her research interest is cloud computing security and cryptography.